



# Actividad n° 2

## Creación de resolver

Integrante: Matias Rojas V.

Profesor: Ivana Bachmann.

Auxiliares: Julián Ferreira.  
Cristian Romero V.

Fecha de entrega: 16 de abril de 2026  
Santiago, Chile

## 1. Configuración Inicial y Tipo de Socket

Para la realización de esta actividad, el resolver fue ejecutado en una máquina virtual asociando el servicio a la IP de la misma (192.168.1.109) y utilizando el puerto 8000 para evitar conflictos con los puertos reservados del sistema.

### ¿Qué tipo de socket debe usar?

Socket "NOC", para realizar consultas rápidas y sin conexión previa. Por lo tanto, el socket adecuado en Python es del tipo Datagrama (`socket.SOCK_DGRAM`)

## 2. flujo del resolver

1. **Escucha y Recepción:** El servidor se enciende y se queda escuchando en el puerto 8000. Cuando recibe una consulta (con dig), recibe el paquete con la pregunta (el dominio).
2. **Inicio de la Búsqueda:** La función principal `resolver()` toma ese dominio y le envía la pregunta al **Servidor Raíz** (192.33.4.12).
3. **El Proceso Recursivo (Las 3 opciones):** Al recibir la respuesta del servidor, el programa decide qué hacer:
  - **Opción A (Éxito):** El servidor le entrega la IP definitiva en la sección **Answer**. El trabajo termina.
  - **Opción B (Delegación con IP):** El servidor no entrega la respuesta, pero delega la consulta a otro servidor. El programa repite la consulta usando la nueva IP. y le entrega su IP (**Authority + Additional**). El programa repite el proceso consultando a esa nueva IP.
  - **Opción C (Delegación sin IP):** El servidor le da el nombre de otro servidor, pero no su IP. El programa hace una "pausa", busca recursivamente la IP de ese nuevo servidor y, cuando la encuentra, retoma su búsqueda original.
4. **Entrega Final:** Una vez que obtiene la IP del dominio solicitado, empaqueta la respuesta en el formato oficial de DNS y se la devuelve al cliente que hizo la pregunta original.

## 3. Pruebas de Funcionalidad

Las pruebas de funcionalidad base demuestran que el resolver procesa correctamente las respuestas, logrando transformar los mensajes a las estructuras correspondientes y consultar iterativamente.

### 3.1. Resolución de dominios

Se verificó la resolución del dominio `www.uchile.cl` y `cc4303.bachmann.cl`, logrando obtener la dirección IP de forma exitosa mediante los siguientes comandos:

- `dig -p8000 @192.168.1.109 www.uchile.cl` resolvió exitosamente la IP 200.89.76.36.
- `dig -p8000 @192.168.1.109 cc4303.bachmann.cl` resolvió exitosamente la IP 104.248.65.245.

### 3.2. Funcionamiento del Caché

Se implementó un sistema de caché para almacenar las últimas consultas. Esto se evidenció repitiendo la consulta al dominio `eol.uchile.cl`:

- **Primera consulta:** El resolver consultó la jerarquía completa (Raíz -> TLD .cl -> Autoritativo .cl), tardando 336 ms.

- **Segunda consulta:** Al repetir el comando, la respuesta se entregó inmediatamente desde el almacenamiento local en **0 ms**, y el modo **debug** mostró el mensaje: Respondiendo desde cache para `eol.uchile.cl`, confirmando el uso del caché.

## 4. Experimentos y Observaciones

### 4.1. Experimento 1: `www.webofscience.com`

Al intentar resolver el dominio `www.webofscience.com`, se observó lo siguiente:

**¿Resuelve el programa este dominio?** No.

**¿Qué sucede?**

La terminal de `dig` reporta múltiples errores `communications error to 192.168.1.109#8000: timed out` y finalmente indica que no se pudo alcanzar ningún servidor.

**¿Por qué sucede esto?**

Dominios grandes y complejos como **Web of Science** rara vez apuntan directamente a un registro A. Usualmente, la respuesta en la sección **Answer** contiene un alias (registro CNAME). Dado que las reglas de nuestro resolver casero establecen explícitamente: **“Si recibe algún otro tipo de respuesta simplemente ignórela”**, al recibir un CNAME en lugar de un registro tipo A o una delegación NS, el programa desecha el paquete. El cliente `dig` se queda esperando una respuesta que nunca llega, causando el **timeout**.

**¿Cómo se arreglaría este problema?**

Se debería modificar el código del resolver para que no ignore los registros de otros tipos. Si detecta un registro tipo CNAME en la respuesta, el resolver debería tomar el nuevo nombre de dominio (el alias) y reiniciar recursivamente el proceso de consulta desde el paso inicial para encontrar la IP definitiva.

### 4.2. Experimento 2: `www.cc4303.bachmann.cl`

Se ejecutó el comando para buscar el dominio `www.cc4303.bachmann.cl` y se contrastó con el comportamiento del resolver público de Google.

**Qué ocurre?**

Al igual que en el experimento anterior, nuestro resolver sufre un **timeout** y no logra devolver ninguna información al cliente.

**¿Qué se habría esperado que ocurriera?**

Se esperaba que el servidor respondiera rápidamente indicando la IP de dicho dominio, en lugar de quedarse atascado o en silencio.

**Contraste con 8.8.8.8 y explicación DNS:**

Al consultarle al servidor 8.8.8.8, este responde de inmediato con un estado NXDOMAIN (Non-Existent Domain) y entrega un registro SOA en la sección **Authority**. Esto indica formalmente que el subdominio con “`www`” no está registrado en esa zona. Nuestro resolver experimenta un **timeout** porque no está programado para entender o procesar un error NXDOMAIN ni registros SOA; está construido estrictamente para buscar delegaciones (NS) en **Authority** y registros A en **Answer/Additional**. Al no encontrar ninguno, ignora la respuesta.

### 4.3. Experimento 3: Consulta iterativa de Name Servers

A través del modo **debug**, se observaron las distintas delegaciones para llegar a un dominio específico mediante varias consultas repetidas.

### ¿Son siempre los mismos Name Servers?

Sí, en todas las iteraciones se siguió el mismo camino: Servidor raíz . -> cl2-tld.d-zone.ca. -> bachmann.cl. -> etc.

### ¿Por qué sucede esto?

Se debe a la naturaleza jerárquica y estricta del protocolo DNS. La resolución de un nombre siempre fluye de derecha a izquierda por el árbol DNS (desde el nivel raíz hacia abajo). La administración de quién tiene la autoridad para cada zona es fija. Por lo tanto, el TLD encargado de .cl y los **Nameservers** autoritativos configurados por el dueño del dominio bachmann.cl serán exactamente los mismos, siempre y cuando el administrador del dominio no cambie deliberadamente su proveedor de infraestructura DNS

## 5. funciones

**Clase DNSCache:** Administra el almacenamiento local para agilizar consultas repetidas. Mantiene un historial circular de las últimas 20 consultas y conserva en memoria únicamente las direcciones IP de los 3 dominios consultados con mayor frecuencia.

- Incluye métodos para registrar nuevas consultas (add\_query), recalcular las frecuencias (update\_cache) y recuperar IPs almacenadas (get).

**send\_DNS\_msg(query\_name, address, port):** Se encarga de la comunicación de red. Construye la pregunta DNS, abre un socket NOC, envía el paquete al servidor indicado y retorna la respuesta ya parseada como un objeto DNSRecord.

**parse\_DNS\_msg(dns\_msg):** Transforma la respuesta bruta (DNSRecord) en un diccionario json, más fácil de manipular. Separa y organiza las secciones fundamentales del mensaje DNS: la consulta (Qname), las respuestas (Answer), los servidores con autoridad (Authority) y los registros adicionales (Additional).

**rr\_to\_dict(rr):** Es una función auxiliar que toma un registro de recurso (Resource Record) individual y lo convierte en un diccionario, extrayendo campos clave como el nombre (rname), el tipo (rtype) y los datos (rdata, que suele ser la IP).

**resolver(msg\_query, server\_name, server\_addr, from\_cache):** Es el motor recursivo principal del programa. Ejecuta la lógica central del protocolo:

- Revisa si el dominio ya está en la caché para responder instantáneamente.
- Si no está en caché, envía la consulta al servidor especificado (iniciando en el servidor raíz).
- Caso Base: Si la respuesta contiene registros tipo A en la sección Answer, almacena la primera IP en caché, empaqueta la respuesta final y la retorna.
- Caso Delegación Directa: Si recibe registros NS en la sección Authority y sus respectivas IPs en la sección Additional, toma la primera IP disponible y se llama a sí misma recursivamente para consultar a ese nuevo servidor.
- Caso Delegación Indirecta: Si recibe registros NS pero no vienen acompañados de IPs en la sección Additional, realiza una sub-consulta recursiva para averiguar primero la IP del Name Server, y luego retoma la consulta original.

**(\_\_main\_\_):** Inicializa el socket NOC en la IP de la máquina virtual y el puerto 8000. Ejecuta un bucle infinito que escucha peticiones de clientes, las deriva a la función resolver() y devuelve los bytes de la respuesta final al cliente original.